

# Seminar 2

## Object-Oriented Design, IV1350 Seminar 2

Lucas Burgos, [lburos@kth.se](mailto:lburos@kth.se)

12 april 2026

### 1. Introduction

This report presents the design of a program for the Repair Electric Bike scenario in the course Object oriented design IV1350. The purpose of this report is to showcase my solution Seminar 2, which should demonstrate the requirements of high cohesion, low coupling and high encapsulation. This report focuses on the design of the program rather than the implementation of the code. The solution is intended to handle the basic flow, the alternative flows and how the program should be started. This report includes a description of my method, the resulting class diagrams and communication diagrams (a type of interaction diagram) and finally a discussion to adress some thoughts and evaluations of the design.

### 2. Method

The design of the class diagram and the communication diagrams was developed iteratively based on the requirements in the Repair Electric Bike scenario. The focus of seminar two was to cover the basic and alternative flows and the startup of the application. I used the MVC and layer patterns and continuously had the concepts of high cohesion, low coupling and good encapsulation in mind while designing.

The first thing I did was to review the specifications from seminar 1 to make sure everything was in order before I begun with seminar two. Thereafter I divided the system into the layers Startup, View, Controller, Integration and Model to separate responsibilities and reduce coupling between the unrelated parts of the system. The Startup-layer is used for dependency setup and object creation, the view layer for user interaction, the controller layer to coordinate operations across the system, the integration layer for registries and printing and finally the model layer for the domain concepts like RepairOrder, DiagnosticReport and RepairTask. With the help of the course literature, I tried to follow the concepts and guidelines for MVC and layers to avoid having the different parts of the system, like user interface code and business logic, in the same classes.

Thereafter I created one system operation at a time using Communication diagrams. I created a Communication diagram for startup, customer search, repair order creation, registration of diagnostic information, preparation for approval, acceptance of repair order and rejection of a repair order. During this process, I tried to follow the UML conventions for Communication diagrams, such as numbering and object identification, and I also implemented guards where alternative flows were needed, such as in the Communication diagram named

"SearchCustomerByPhoneNr" (Figure 2). While creating the Communication diagrams I updated the class diagram for each new Communication diagram. Classes was added and associations were drawn. By doing this, the class diagram could be seen as a summary of the system.

During the design I made some decisions with the ambition to increase cohesion and encapsulation. One example of this is that the controller is limited to coordination responsibilities, while state changes, such as marking a repair order as accepted or rejected, are placed in the RepairOrder class. Another is that I gradually refined the model to use objects rather than primitive values, like representing task cost with the class Amount and allowing repairOrder to calculate the total cost. This makes the parameter lists short and the data is kept together inside domain-objects.

The UML editor I used to create the Com- and Class diagrams was Astah. While working, I continuously revised the diagrams to improve naming, message order, parameter types and responsibility.

### 3. Result

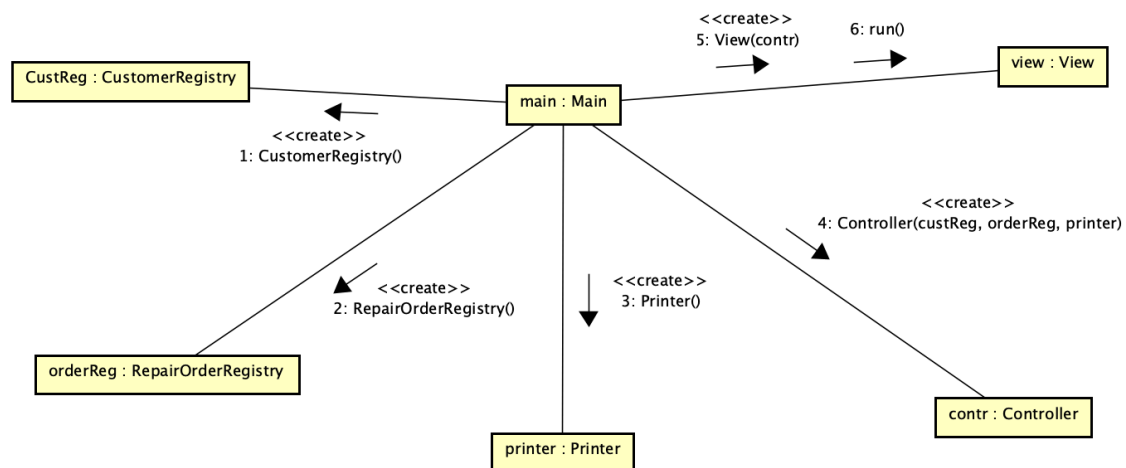


Figure 1

#### 3.1 Startup of the application

The startup of the application is shown in in figure 1. In this diagram, the main method creates the necessary objects for the program to run. These are

- CustomerRegistry: the register for all customers
- RepairOrderRegistry: the register for the repair orders
- Printer: responsible for printing the order

- Controller: responsible for connecting the user interface and handling the operations therefrom.
- View: responsible for the user interface

The Controller receives references to the register-objects and the printer, while the view receives a reference to the controller. After the objects have been created and the dependencies has been established, the main method calls run() in the view to start the program.

### 3.2 Search for customer by phone number

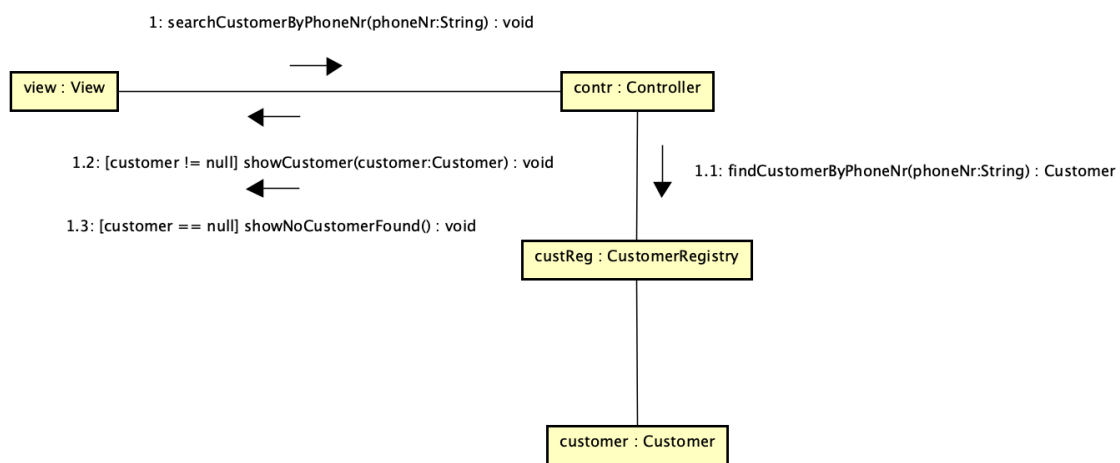


Figure 2

Figure 2 shows the operation of searching for an existing customer. The user of the program enters a phone number in the view, which sends the search to the controller. The controller delegates the task to the method findCustomerByPhoneNr in the CustomerRegistry. The method searches to find a match, and then returns the result. If a customer is found, the customer details is shown in the view. If not, the view informs that a match could not be found with the given phone number and therefore cannot continue to create a repairOrder. This covers both the normal outcome and the alternative flow for an unknown phone number.

## 3.3 Creation of a repair order

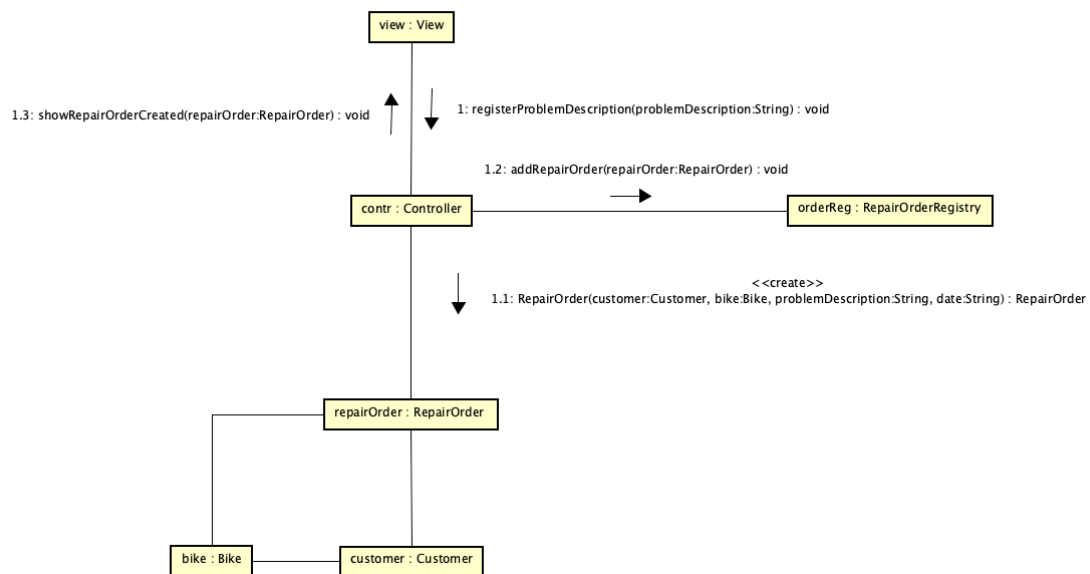


Figure 3

Figure 3 shows the creation of a repair order. After the customer and bike details have been retrieved and confirmed in the previous customer search, the user enters the problem description. The view then sends the problem description to the controller. The controller uses the previously retrieved customer and bike when creating the RepairOrder. The date is set when the RepairOrder is created. The created repair order is then stored in the RepairOrderRegistry and finally presented in the view.

## 3.4 Register a diagnostic report and proposed repair task

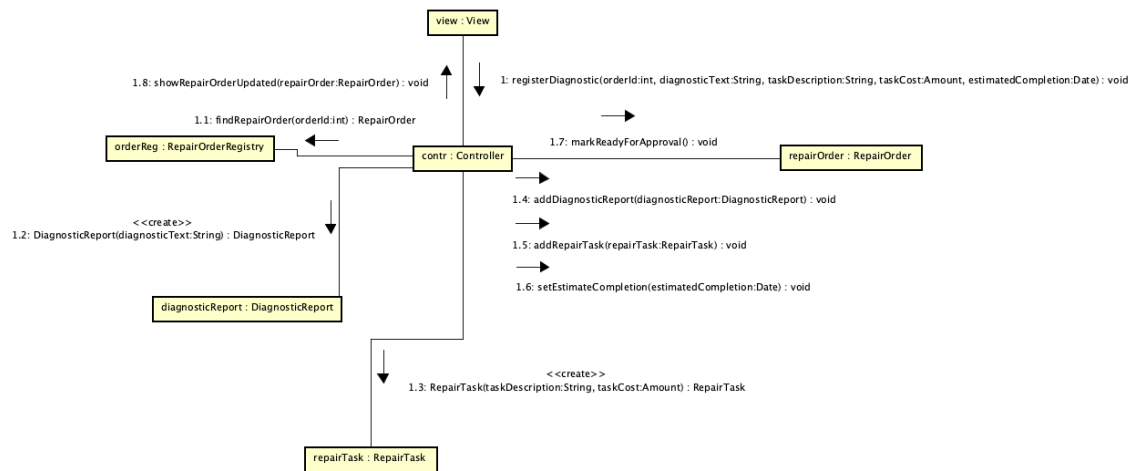


Figure 4

Figure 4 shows the registration of diagnostic information. First, the technician inputs the id, the diagnostic text, the task description, the taskCost and the estimated completion date with the help of the method `registerDiagnostic`. The view then sends everything to the controller. The controller then retrieves the repair order and then creates a `DiagnosticReport` and `RepairTask`, which are added to the `repairOrder`. The estimated completion date is also set for the repair order. Finally, the `repairOrder` is marked as ready for approval and the updated `RepairOrder` is presented in the view.

## 3.5 Approval and acceptance of repairOrders

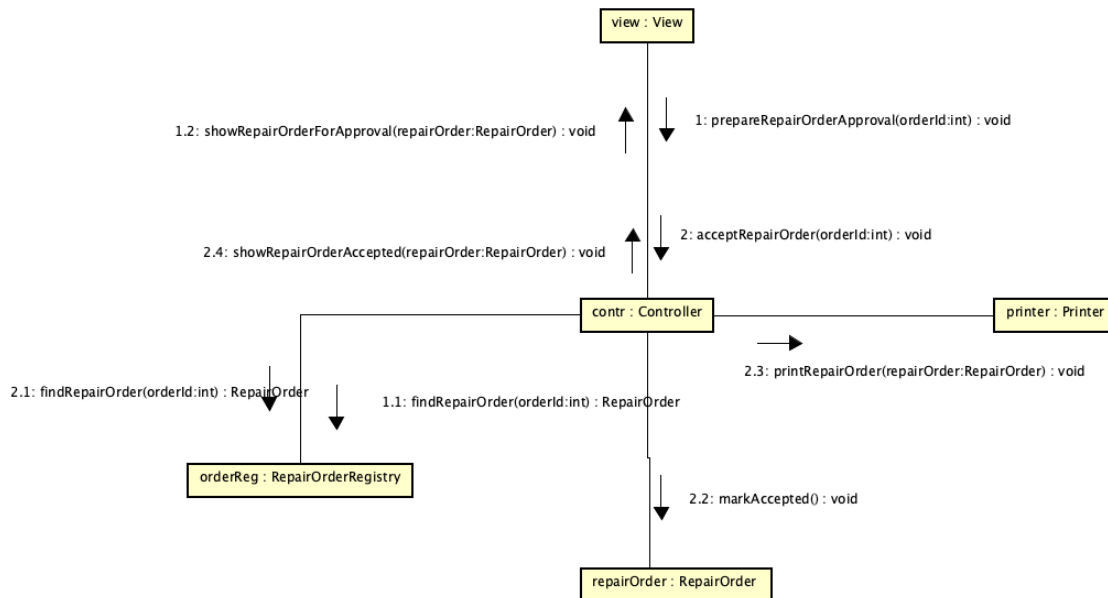


Figure 5

Figure 5 displays the communication diagram of approving a repairOrder which is marked as ready for approval in the previous flow. The interaction begins when the user inputs the orderId in the view. The view then sends the user request to the controller which retrieves the repairOrder from repairOrderRegistry. The repairOrder is then shown in the view which shows the diagnostic report, the proposed repair task, the task cost, the estimated completion date and the total cost of the repair order. If the user likes the proposal, the user accepts the repair order. The view then sends an update request via acceptRepairOrder to the controller. The controller then retrieves the specified repairOrder, marks it accepted and sends it to the printer to print out a receipt. Finally, the accepted repairOrder is presented in the view.

## 3.6 Rejection of repairOrder

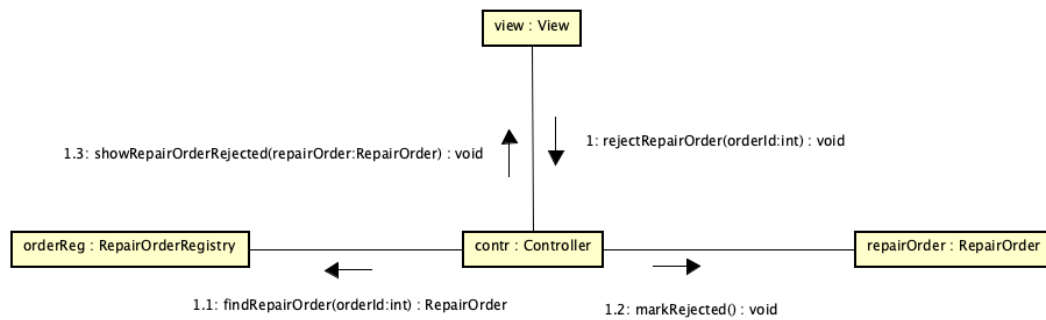


Figure 6

Figure 6 shows the alternative flow where the customer does not accept the proposed repair order. When the user rejects the repair order, the view sends a `rejectRepairOrder` message to the controller with the `orderId`. The controller retrieves the specified `RepairOrder` from the `RepairOrderRegistry` and marks it as rejected. No deletion is performed since the requirements states that repair orders should remain in the registry even when rejected.

## 3.7 The class diagram

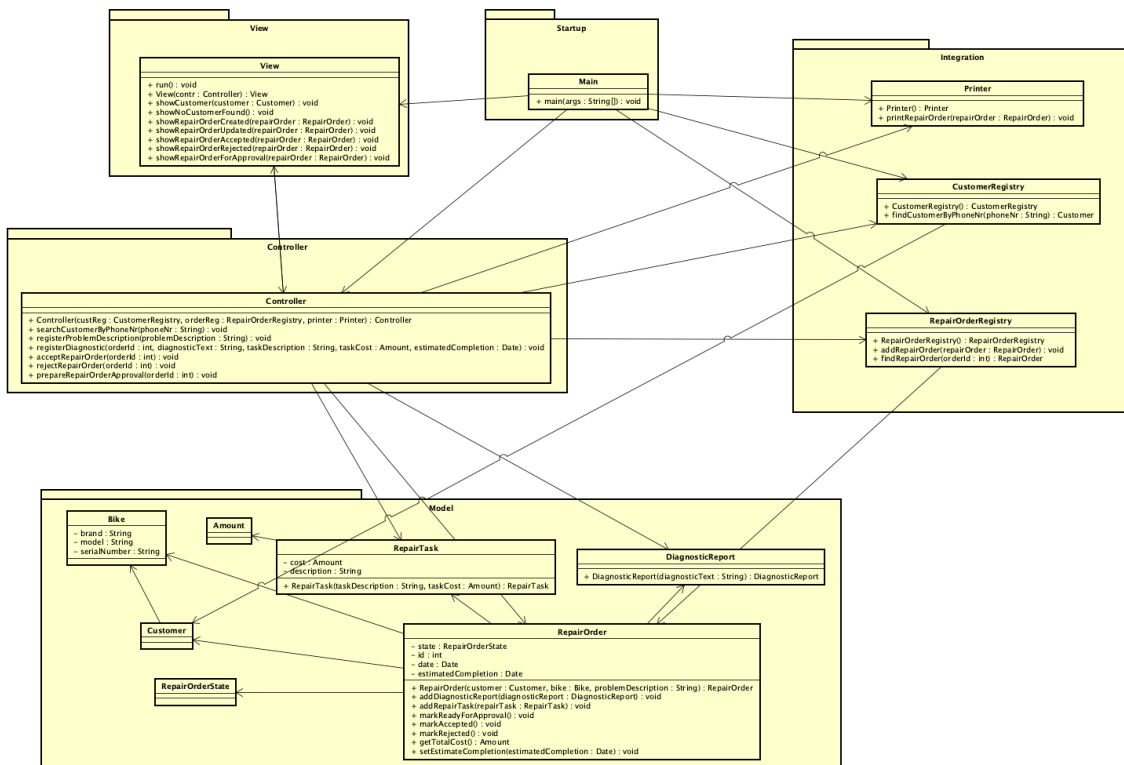


Figure 7

Figure 7 shows the complete class structure of the design. The class diagram includes all classes that appear in the interaction diagrams, as required by the assignment. The diagram shows the layer structure and the main dependencies between the layers. View depends on Controller, while controller depends on the integration-classes Printer, CustomerRegistry and RepairOrderRegistry, as well as the relevant model classes such as RepairTask, DiagnosticReport and RepairOrder. RepairOrder is associated with customer, Bike, DiagnosticReport, RepairTask and RepairOrderState. In the Model layer, RepairOrder is the central domain class. Customer is associated with Bike, and each RepairTask contains both a description and a cost represented by Amount. RepairOrder contains the date when it was created as well as the estimated completion date. The Model also includes behaviour for changing its state, for example to ready for approval, accepted and rejected, as well as behaviour to set and obtain the cost. Together, these classes represent the domain concepts required according to the specifications.

## 3.8 Summary.

The class diagram summarises the structural result of the design, while the communication diagrams explain how the objects collaborate during different system operations. Together, these diagrams describe the full design of the repair electric bike application.



## 4. Discussion

The design was built using the task description, assessment criteria of seminar 2, and the course literature. The discussion below is an evaluation of my solution in relation to the principles given by the course lectures and the literature such as the MVC and layer framework, low coupling, high cohesion and good encapsulation.

A clear strength of the design is the use of layers Startup, View, Controller, Integration and model mentioned in the MVC framework. This follows the structure given by the course literature. In the solution, the Startup layer is only used for object creation and dependency startup, the View handles the user interaction, the Controller coordinates system operations, the integration handles registries and printing and the model contains the domain objects. This separation makes the design easier to understand, where each layer has a clear purpose. This separation makes the design easier to expand.

The design shows a relatively high level of cohesion since most classes have one main responsibility. For example, CustomerRegistry is responsible for searching for a customer with the method findCustomerByPhoneNr, RepairOrderRegistry stores and retrieves repair orders, Printer handles printing of the accepted repair order, and View presents information to the user. In the model layer, RepairOrder is the central domain class and is responsible for holding and updating its own state. Methods like addRepairTask, markAccepted and markRejected has their responsibility in the domain model instead of In the controller. This improves cohesion both in the controller and in the model, since the controller mainly coordinates operations while the model handles domain-specific behaviour.

Low coupling has been a high priority throughout the design but its also the point where there is room for different perspectives. One concern is that the controller is almost too central according to the pure definition of coupling, but it should be according to the MVC framework. Discussing this with my professor of the course, Leif Lindbäck, gave a good perspective. He mentioned a rule of thumb where 4 connections from the controller to other classes in the communication diagrams are good, but more than that should be reconsidered by instead increasing the encapsulation. This is not an absolute rule, but it made me think since I sometimes have 4 connections between my controller and other classes in my communication diagrams. Is it worth it to decrease the connection by one and instead add a class before the two previous ones? My conclusion is that it is not since its just four, and having less but increasing encapsulation leads to a higher degree of complexity which might not be necessary. Although when five or six connections might emerge later on, this could be a good solution. This could lead to another perspective regarding Main. Main creates the registers, the printer, the controller and the view directly. This is a simple design, however an alternative solution would be to introduce a separate class RegistryCreateor which creates all registries necessary. This would reduce coupling from main and increase encapsulation in the startup part. On the other hand, this increases the complexity.

Encapsulation is another principle which has been used. Instead of using primitive data like individual strings and ints, the design has instead tried to use objects where applicable. The course literature warns against excessive use of primitive data and unnecessarilylong paramterlists since it weakens encapsulation and makes the design harder to maintain when expanding it. In this design, classes such as RepairOrder, Customer and Bike are used to represent central concepts in the program. A clear example of this is using the object Amount

for task cost, which makes the approval flow more consistent with the scenario where the customer must be informed about proposed repair tasks.

The communication diagrams and the class diagrams are fairly consistent with one another. The interaction diagrams describe the startup, customer search, repair order creation, registration of diagnostic information, preparation for approval, acceptance and rejection. This structure follows the seminar requirement that the functionality shall be described in the interaction diagrams. The class diagram then summarises the result of those interaction diagrams by showing the layers, classes, methods, attributes and associations.

In summary, the design should satisfy the seminar 2 assessment criteria. It has a clear layer structure, the communication diagrams cover the functionality and the model contains the domain behaviour.

## References

Lindbäck, L. (2026). A First Course in Object-Oriented Development: A Hands-On Approach. January 14, 2026 version.

IV1350 Seminar 2 task list

IV1350 Seminar 2 assessment criteria

Declaration on the use of AI: I used ChatGPT to support my design work and the report writing. The AI was used to explain design concepts from the course literature, debate alternative ways of structuring the system and to suggest improvements related to coupling, cohesion, encapsulation and object responsibilities. I verified the AI's output by comparing it with the seminar 2 assignment instructions, the assessment criteria, the Repair Electric Bike scenario and the course literature.